

PLC App

PLC Runtime Environment for ctrlX OS 03VRS

Copyright

© Bosch Rexroth AG 2025-08

All rights reserved, also regarding any disposal, exploitation, reproduction, editing, distribution, as well as in the event of applications for industrial property rights.

Disclaimer

The specified data only serves to describe the product. As our products are constantly being further developed, no statements concerning a certain condition or suitability for a certain application can be derived from our information. The information given does not release the user from the obligation of own judgment and verification. Please note that our products are subject to a natural process of wear and aging.

DOK-XCORE*-PLC***V03**-AP03-EN-P

DC-AE/PAG-SW (MiNi)/(PiaSt)

Table of contents

1	About this documentation	5
2	Important directions on use	5
2.1	Intended use	5
2.1.1	Introduction	5
2.1.2	Areas of use and application	6
2.2	Unintended use	6
3	Safety instructions	7
4	Introduction and overview	7
4.1	PLC App – Basics	7
4.2	PLC App – Basics	10
5	Target systems	12
6	Data Layer connection	13
7	Multicore functionality	14
7.1	Introduction	14
7.2	Licensing the multicore functionality	15
7.3	Licensing the multicore functionality	16
7.4	Multicore in the IEC program	16
7.4.1	Configuration of the PLC tasks	16
7.5	Configuration of the PLC tasks	19
8	Flexible configuration of the memory for remanent PLC data	22
9	Connecting the PLC runtime system to the ctrlX OS system behavior	23
10	Optimized access to complex IEC data types via the Data Layer	24
10.1	Using structures	24
10.2	Using arrays of structures	26
10.3	Using structures with bits	28
10.4	Using enums	32
10.5	Method calls	35
10.6	Setting options	37
10.6.1	Enum representation	37
10.6.2	Browsing array variables via OPC UA	39
10.7	Restrictions	39
11	Task configuration	39
11.1	General information on task configuration	39
11.2	Event sources for external event-controlled PLC tasks	40
11.3	Support of the automatic program configuration of the ctrlX scheduler	40
11.4	Freely configurable, external event sources	41
12	Creating external POU's	42
13	ctrlX UI – Elements	42
13.1	Navigation	42
13.1.1	Side navigation – PLC App	42
13.2	Windows	44
13.2.1	Window – “PLC”	44
13.3	Header	45
13.3.1	Header – “PLC”	45

13.4	Tabs	45
13.4.1	Tab – “Applications”	45
13.4.2	Tab – “Boot applications”	46
13.4.3	Tab – “Project information”	47
14	Related documentation	48
14.1	Overview	48
14.2	ctrlX AUTOMATION	48
14.3	ctrlX WORKS	49
14.4	ctrlX OS	49
14.5	ctrlX OS apps	50
15	Service and support	54
16	Index	55

1 About this documentation

Editions of this documentation

Edition	Date	Note
01	2024 – 09	First edition of the PLC App Version PLC 3.2
02	2025 – 02	Edition for PLC App Version PLC 3.4 Revised contents: ➔ PLC App – Basics ➔ Tab – “Applications”
03	2025 – 08	Edition for PLC App Version PLC 3.6 Revised contents: ➔ Multicore in the IEC program ➔ Optimized access to complex IEC data types via the Data Layer New content: ➔ Task configuration

2 Important directions on use

2.1 Intended use

2.1.1 Introduction

Rexroth products are developed and manufactured to the state-of-the-art. The products are tested prior to delivery to ensure operational safety and reliability.

⚠ WARNING

Personal injury and damage to property due to incorrect use of products!

The products may only be used as intended.

Failure to use the products as intended may cause situations resulting in property damage and personal injury.

NOTICE**Damages resulting from unintended use**

Rexroth As the manufacturer does not assume any warranty, liability or compensatory claims for damages resulting from unintended use of the products. The user alone shall bear the risks of an unintended use of the products.

Before using Rexroth products, make sure that all the prerequisites for an intended use of the products are met:

- Personnel that in any way, shape or form uses Rexroth products must first read and understand the relevant safety instructions and be familiar with their intended use
- Leave hardware products in their original state, i.e., do not make any structural modifications. It is not permitted to decompile software products or alter source codes
- Do not install damaged or defective products or commission them
- It has to be ensured that the products have been installed as described in the relevant documentation

2.1.2 Areas of use and application

Products of the ctrlX series are suitable for Motion/Logic applications.

NOTICE

Products of the ctrlX series may only be used with the accessories, mounting parts, and other components specified in this documentation. Components that are not expressly mentioned must neither be attached nor connected. The same applies to cables and lines.

Only to be operated with the hardware component configurations and combinations expressly specified and with the software and firmware specified in the corresponding documentations and functional descriptions.

Products of the ctrlX series are suitable for single-axis as well as for multi-axis drive and control tasks. Device types with different equipment and interfaces are available for using the system in specific applications.

Typical areas of application:

- Building automation
- IoT and Security Gateway or Device
- Handling & Robotic

Controls of the ctrlX CORE series may only be operated under the mounting and installation conditions, in the position of normal use and under the ambient conditions (temperature, degree of protection, humidity, EMC, etc.) specified in the related documentations.

2.2 Unintended use

"Unintended use" refers to using the ctrlX products outside of the above-mentioned areas of application or under operating conditions and technical data other than described and specified in the documentation.

ctrlX products must not be used if they are exposed to following conditions:

- Operating conditions that do not meet the specified ambient conditions. Operation under water, under extreme temperature fluctuations or under extreme maximum temperatures is prohibited
- Applications that have not been expressly authorized by Rexroth

3 Safety instructions

The Safety instructions contained in the available application documentation feature specific signal words (DANGER, WARNING, CAUTION or NOTICE) and, where required, a safety alert symbol (in accordance with ANSI Z535.6-2006).

The signal word is meant to draw the reader's attention to the safety instruction and identifies the hazard severity.

The safety alert symbol (a triangle with an exclamation point), which precedes the signal words DANGER, WARNING and CAUTION, is used to alert the reader to personal injury hazards.

The Safety instructions in this documentation are designed as follows:

 DANGER	In case of non-compliance with this safety instruction, death or serious injury will occur.
 WARNING	In case of non-compliance with this safety instruction, death or serious injury could occur.
 CAUTION	In case of non-compliance with this safety instruction, minor or moderate injury could occur.
NOTICE	In case of non-compliance with this safety instruction, property damage could occur.

4 Introduction and overview

4.1 PLC App – Basics

The installation of the "PLC App", the functional scope of the ctrlX CORE control is supplemented by a PLC runtime environment

The runtime environment is based on "CODESYS Control V3" and is completely integrated into the ctrlX OS system environment.

PLC programming and debugging is carried out using the PC-based engineering tool ctrlX PLC Engineering.

PLC App Licensing

The operation of the PLC App is subject to licensing.

The license model contains a total of four licenses, which can be used to scale the scope of functions.

The "PLC Basic" license is required to operate the PLC App.

Based on the PLC Basic license, the scope of functions can be extended by adding upgrade licenses.

Licenses/scope of functions

Licenses	PLC tasks	CPU cores	I/O data
PLC Basic	1	1	Up to 256 bytes
PLC Standard	Unlimited	1	Up to 1024 bytes
PLC Advanced	Unlimited	Unlimited	Up to 2048 bytes
PLC Performance	Unlimited	Unlimited	Unlimited

Licenses/ordering information

License	Description
PLC Basic	Basic license for operating the PLC App on the ctrlX device Type designation: SWL-XC*-PLC-PLC*XOS*****-BANN Part number: R911421866
PLC Standard	Upgrade of the PLC Basic license Requires the following license on the ctrlX device: • PLC Basic Type designation: SWL-XC*-PLC-PLC*XOS*****-NNNN Part number: R911421867
PLC Advanced	Upgrade of the PLC Standard license Requires the following licenses on the ctrlX device: • PLC Basic • PLC Standard Type designation: SWL-XC*-PLC-PLC*XOS*****-ADNN Part number: R911421868
PLC Performance	Upgrade of the PLC Advanced license Requires the following licenses on the ctrlX device: • PLC Basic • PLC Standard • PLC Advanced Type designation: SWL-XC*-PLC-PLC*XOS*****-PRNN Part number: R911421869

Automatic license check

Each time a PLC application is loaded (boot project, download, load configuration), the PLC automatically performs the following steps to check the license:

1. ➤ Reading out the licenses that are available on the ctrlX device (Basic, Standard, Advanced, Performance)
2. ➤ Recording of the application properties and comparison with the existing licenses

Test points:

- Number of tasks: Single/multi task
- Number of CPU cores used
- Number of I/Os used in the project nodes:
 - Realtime_Data (ethercat_master_instances_ethercatmaster)
 - CODESYS ProfiNet Controller/Device
 - CODESYS EtherNET/IP Scanner/Adapter
- ➔ If the license is correct, the project can be loaded into the PLC and switched to the "RUN" operating state.
- ➔ Reactions in case of insufficient licensing:
 - The download of an application is rejected.
 - Switching to "RUN" operating status is denied.
 - Error message in the ctrlX OS diagnostic system.
 - Error message in the plc logger.
- ➔ Reaction if licenses expire during runtime:
 - Functions of the expired license are stopped

Determine required licenses in the PLC project

ctrlX PLC Engineering provides the following functions to determine the required licenses:

- **Determine required licenses in the project**
The "Licensed Software Metrics" tab in the device editor of the control node shows the parameters to be taken into account for licensing in the project.
- **Comparison of existing and required licenses**
If the control is connected, the "*Status* → *Licenses*" tab lists all existing and all required licenses.

License check for virtual ctrlX devices

Virtual ctrlX CORE devices are subject to a license-free runtime limit of 4 hours. At the end of the runtime, the control shuts down automatically, but can be restarted and operated for 4 more hours. Installed add-ons from Codysys define their own time period depending on the individual configuration.

Additional information

Documentation

- ➔ [ctrlX OS - App Basics](#)
- ➔ [ctrlX OS - License overview](#)

Sources of supply

- ➔ [ctrlX STORE](#)

Web links

- [↗ ctrlX AUTOMATION - Community](#)
- [↗ ctrlX AUTOMATION - How-to](#)
- [↗ ctrlX AUTOMATION - Forum](#)

4.2 PLC App – Basics

The installation of the “PLC App”, the functional scope of the ctrlX CORE control is supplemented by a PLC runtime environment

The runtime environment is based on "CODESYS Control V3" and is fully integrated into the ctrlX CORE system environment.

PLC programming and debugging is carried out using the PC-based engineering tool ctrlX PLC Engineering.

PLC App licensing

The operation of the PLC App is subject to licensing.

The license model contains a total of three licenses, which can be used to scale the scope of functions as required.

The "Basic Single License" is required to operate the PLC App.

Based on the "Basic Single License", the scope of functions can be extended by adding "In-App" licenses.

License name	Description	Type/material number
ctrlX PLC Basic Single license	Basic license to operate the PLC App • Operation of a PLC task on the target device	SWL-XC*-PLC-PLC*****-BANN R911397817
ctrlX PLC Standard Single license	Additional in-app license <u>Function extension:</u> • Operation of several PLC tasks on the target device possible <u>Requires:</u> • ctrlX PLC Basic Single license	SWL-XC*-PLC-PLC*****-NNNN R911411765
ctrlX PLC Advanced Single license	Additional in-app license <u>Function extension:</u> • Operation of several PLC tasks on the target device possible • Distribution of tasks to several CPU cores possible <u>Requires:</u> • ctrlX PLC Basic Single license • ctrlX PLC Basic Standard Single license	SWL-XC*-PLC-PLC*****-ADNN R911401461

Automatic license check

Each time a PLC application is loaded (boot project, download, load configuration), the PLC automatically performs the following steps to check the license:

1. Read out the licenses and check which PLC licenses are available on the ctrlX device (Basic, Standard, Advanced, Performance)
2. Check of the application properties (single task, multi task, multi core) and comparison with the existing licenses
 - ➔ If the license is correct, the PLC can be switched to the "RUN" operating state
 - ➔ Reactions in case of insufficient licensing:
 - Switching to "RUN" operating status is denied
 - Error message in the ctrlX-CORE diagnostic system
 - Error message in the plc logger
 - ➔ Reaction if licenses expire during runtime:
 - Functions of the expired license are stopped

Exceptions to the automatic license check

An exception in case of a missing licensing is the "ctrlX PLC Advanced Single" license, which allows the application to be started even if the license is missing.

In this case, however, the application is automatically stopped after 30 minutes.

License check for virtual ctrlX devices

Virtual ctrlX CORE devices are subject to a license-free runtime limit of 4 hours. At the end of the runtime, the control shuts down automatically, but can be restarted and operated for 4 more hours. Installed add-ons from Codsys define their own time period depending on the individual configuration.

Authorization concept

The authorization concept for the PLC app provides the following authorizations for access to PLC-relevant data:

Authorization	ctrlX OS web interface	ctrlX PLC Engineering
View PLC (PLC view)	Read access to PLC-relevant data (Data Layer node).	Connection to the control not possible.
Operate PLC (PLC operate)	- Read access to PLC-relevant data (Data Layer node). - Restricted write access (start/stop/reset warm/reset cold).	Connection to the control not possible.
Manage PLC (PLC manage)	No restrictions.	No restrictions.

The settings can be assigned for the respective user in the PLC category, see: [ctrlX OS Web Documentation](#)

Please also consider the other categories when assigning user rights.

Additional information**Documentation**

- [↗ ctrlX CORE - Apps basics](#)
- [↗ ctrlX CORE - License overview](#)

Sources of supply

- [↗ ctrlX App Store](#)

Web links

- [↗ ctrlX CORE - Community](#)
- [↗ ctrlX CORE - How-to](#)
- [↗ ctrlX CORE - Forum](#)

5 Target systems

The PLC App supports target platforms based on ARM64 and x64.

Programming in ctrlX PLC Engineering is processor-dependent via the associated devices:

- ctrlX CORE ARM64
- ctrlX CORE x64

The mapping of the ctrlX CORE hardware variants to the respective device is defined in the following table:

ctrlX CORE hardware variant	Device to be selected in ctrlX PLC Engineering
ctrlX CORE X2	ctrlX CORE ARM64
ctrlX CORE X3	ctrlX CORE ARM64
ctrlX CORE X3+	ctrlX CORE ARM64
ctrlX DRIVE +	ctrlX CORE ARM64
ctrlX CORE x7	ctrlX CORE x64
ctrlX CORE virtual	ctrlX CORE x64
ctrlX CORE OS	ctrlX CORE x64

Project templates

When creating a new project in ctrlX PLC Engineering platform-specific project templates are provided for selection:

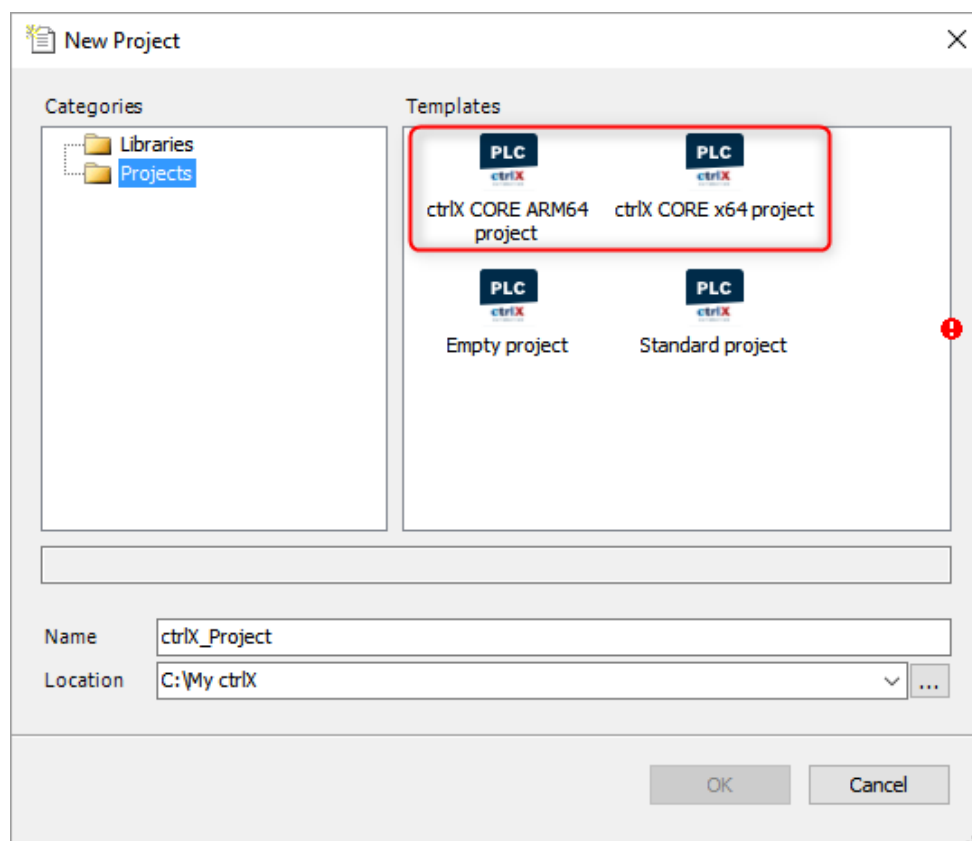


Fig. 1: Dialog for selecting a project template in ctrlX PLC Engineering

Cycle times / memory sizes / boot project

The minimum adjustable task cycle time for all device variants is 1 ms.

The size of the usable retain memory depends on the hardware. The default value is 64 kB, but can be reduced or increased depending on the hardware limits.

The boot project is stored on the target device in specific folders in the configuration (app data).

6 Data Layer connection

The PLC App publishes its relevant data or commands via the Data Layer node “plc”. The sub-nodes reference general and application-specific data and commands.

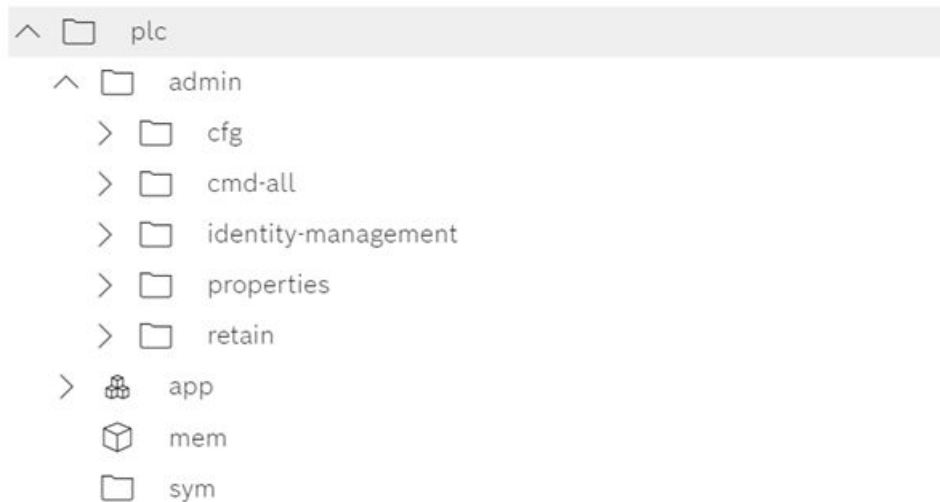


Fig. 2: Node "plc" in the Data Layer of the ctrlX OS web interface

7 Multicore functionality

7.1 Introduction

The multicore function allows several CPU cores of the ctrlX device to be used for parallel processing of PLC IEC tasks.

Advantages:

- Increased execution speed through parallel processing with simultaneous utilization of the available processor power.
- Complex PLC logics can be divided into different, independent task groups and CPU cores.
- Separation of logic application, communication and additional functions such as Motion or visualization.

Requirements

- ctrlX device type from CORE X3 or ctrlX DRIVE plus with integrated ctrlX CORE
- ctrlX version from PLC-V-0110 or higher
- Required licenses on the ctrlX device:
 - PLC Basic
 - PLC Standard
 - PLC Advanced

Automatic check for multicore capability

During the ctrlX device start up, the available processor cores and their properties are determined automatically (real-time or non-real-time). The PLC run-time system queries these properties during start-up and assigns the system tasks to the first available real-time processor core. The execution of the PLC tasks is also assigned to the first available real-time processor core.

Via the task configuration in ctrlX PLC Engineering, the user can also manually assign the execution of the PLC tasks to other available processor cores. The required licenses have to be installed on the ctrlX device for execution, see:

➔ [Licensing the multicore functionality](#)



In ctrlX PLC Engineering, 4 processor cores are always offered for assignment in the task configuration, regardless of the actual properties of the target system. If an application is loaded onto a target system whose processor configuration cannot be executed on the target device, the download is aborted and an error message is reported.

related topics:

- ➔ Apps and licenses in the ctrlX OS STORE
- ➔ License center "Quick Start Guide"
- ➔ Multicore and task configuration - Basics and configuration

7.2 Licensing the multicore functionality

related topics:

- ➔ License Center "Quick Start Guide"
- ➔ Apps and licenses in the ctrlX OS STORE

The use of multicore functionality on the ctrlX device requires the following licenses:

License	Description	License type code	Material number
PLC Basic	Basic license for operating the PLC app and the PLC runtime system on the ctrlX device	SWL-XC*-PLC-PLC*XOS*****-BANN	R911421866
PLC Standard	Upgrade of the PLC Basic license	SWL-XC*-PLC-PLC*XOS*****-NNNN	R911421867
PLC Advanced	Upgrade of the PLC Basic license	SWL-XC*-PLC-PLC*XOS*****-ADNN	R911421868

The upgrade licenses are installed via the web interface of the ctrlX CORE control, see ➔ [further documentation](#)

Multicore project - behavior without sufficient license

If licensing is insufficient, PLC projects configured for multicore systems are only operated in demo mode for 120 minutes. In this case, processing takes place on just one processor core.



After the 120 minutes have elapsed, the PLC switches into STOP state with an exception and the following message is displayed in the PLC logger: „Multi-core demo mode expired! Reboot of the controller is necessary now!“

Convert existing projects to multicore

If an existing project is to be converted to multicore functionality, please note that the control device description file (device description) has to be version V1.10.0.1 or higher.

For projects that were created with a PLC Engineering version prior to WRK-V-0110, it is necessary to update the device description file of the control, see command ➔ [Update Device](#).

7.3 Licensing the multicore functionality

related topics:

- ➔ [License Center "Quick Start Guide"](#)
- ➔ [Apps and licenses in the ctrlX OS STORE](#)

The use of the multicore functionality on the ctrlX CORE control requires the following license:

Name	Description	License type code	Material number
PLC Advance license	In-app license to activate the PLC multicore functionality on the ctrlX control.	SWL-XC*-PLC-PLC*****_ADNN	R911401461

The PLC Advance license is installed via the web interface of the ctrlX CORE control, see ➔ [further documentation](#)

Multicore project - Behavior without PLC Advance license

If no PLC Advance license is installed on the ctrlX CORE control, PLC projects that are configured for multicore systems are operated in demo mode for 120 minutes. In this case, processing takes place on one processor core.



After the 120 minutes have elapsed, the PLC switches to the STOP state with an exception and the following message is displayed in the PLC logger: „Multi-core demo mode expired! Reboot of the controller is necessary now!“

Convert existing projects to multicore

If an existing project is to be converted to multicore functionality, please note that the controller device description file (device description) has to be version V1.10.0.1 or higher.

For projects that were created with a PLC Engineering version prior to WRK-V-0110, it is necessary to update the device description file of the control, see command ➔ [Update Device](#).

7.4 Multicore in the IEC program

The multicore functionality can result in an increased performance when processing the PLC program by partitioning the IEC tasks and their load to different CPU cores.

In case of projects without multicore functionality, the tasks are processed on CPU kernel # 2.

- User IEC tasks
- Communication tasks
- System tasks

7.4.1 Configuration of the PLC tasks

All tasks that run in the context of the PLC app are processed on the first real-time core of the ctrlX OS control. The multicore option allows the user to configure the IEC tasks configured in the PLC program and distribute them to the available CPU cores. In ctrlX PLC Engineering, the task groups are provided, which are used to assign the available cores:



From version PLC-V-0206, the PLC system tasks (communication,...) can also be assigned to the various cores.

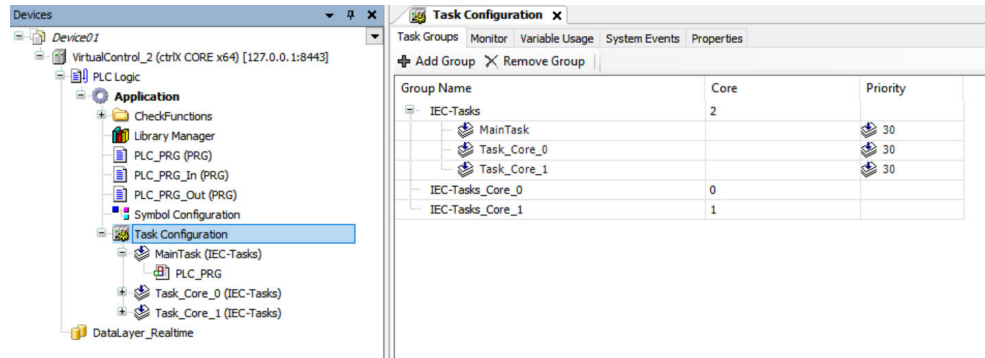


Fig. 3: ctrlX PLC Engineering task configuration

Requirements

- The ctrlX CORE control supports multicore functionality (from device description V1.10.0.1)
- You have defined at least two tasks in your application, for example "MainTask (IEC task)" and "LowTask (IEC task)"

Configuring the assignment

The tasks are assigned to the CPU cores via ctrlX PLC Engineering in the Task Groups tab, see [further documentation](#).

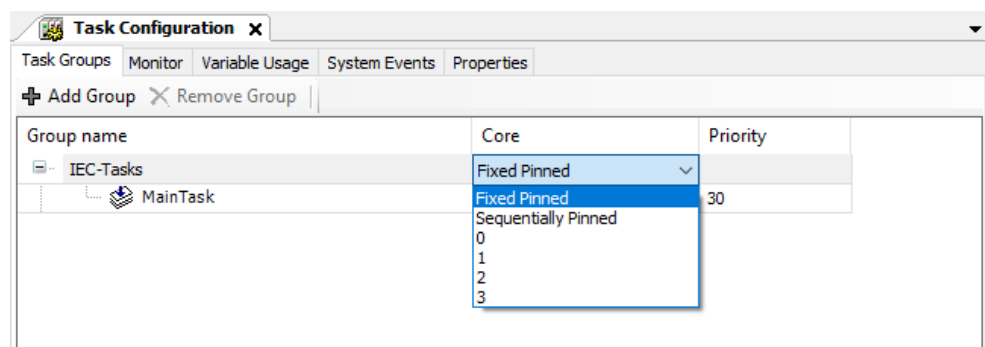


Fig. 4: Task groups tab

In the as delivered condition, the "IEC tasks" task group is already preconfigured in the "Fixed Pinned" setting, in which all tasks in CPU core #2 are processed.

To divide tasks between different cores, first create another task group. Then select the CPU number that is to be responsible for processing the second task group in the "Core" field. Now drag & drop the desired tasks into the new task group.

In the "Sequentially pinned" setting, the CPU cores are assigned sequentially



Please note that the ctrlX OS operating system is running on CPU cores 0 and 1.

If task groups are running on these cores, the watchdog of the IEC task should always be activated.

Functional description

The system defines four (system) task groups to subdivide the functionality:

- **System**

All tasks of the system, such as scheduler, exception handler, etc.

- **Communication**

All communication tasks.

- **Visualization**

All tasks edit the visualization.

- **Asynchronous operations**

All tasks of the asynchronous operations e.g: SysFileAsync

These task groups and their configuration are mapped in the Data Layer on the node: plc->admin->runtime-settings->SysCpuMultiCore'

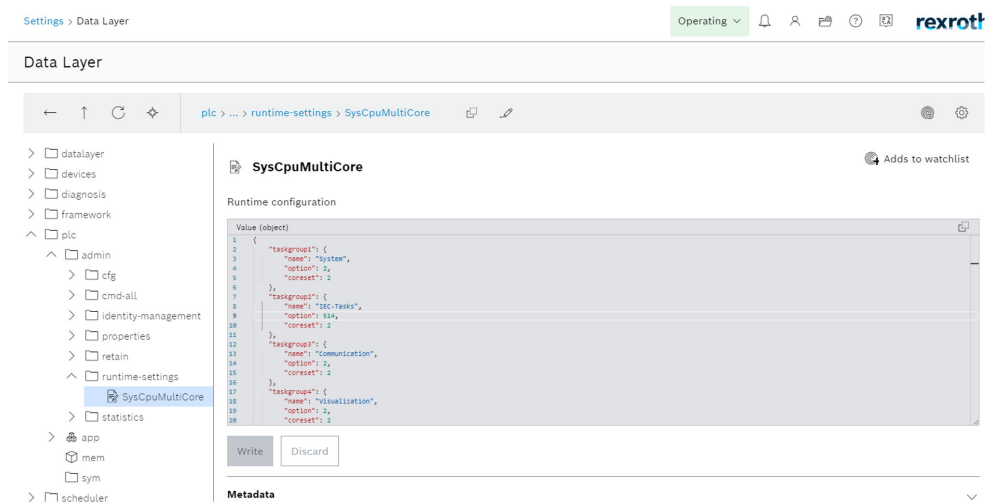


Fig. 5: ctrlX OS – Data Layer

The name and options of the respective task group are fixed and cannot be changed. The “coreset” parameter can be used to select the processor core on which this task group is to be processed. The processor cores available on the ctrlX OS control for this assignment are also mapped under this Data Layer node:

SysCpuMultiCore

Runtime configuration

Value (object)	
22	"taskgroup5": {
23	"name": "Asynchronous Operations",
24	"option": 2,
25	"coreset": 2
26	},
27	"ieccoreset": [
28	0,
29	1,
30	2,
31	3
32	]
33	}
34	

Fig. 6: ctrlX OS – Data Layer / available processor cores



IEC tasks always have a higher priority than the operating system and can slow down the operating system considerably or render it unusable if the load is too high.

Notes

- Changes to the standard configuration affect the runtime behavior of the overall system and are only recommended for experts.
- The ctrlX OS control has to be restarted for the new settings for the task group(s) to take effect.

Further information on the multicore application can be found in the PLC Engineering documentation, see [Multicore and task configuration](#)

7.5 Configuration of the PLC tasks

All tasks that run in the context of the PLC app are processed on the first real-time core of the ctrlX OS control. The multicore option allows the user to configure the IEC tasks configured in the PLC program and distribute them to the available CPU cores. In ctrlX PLC Engineering, the task groups are provided, which are used to assign the available cores:



From version PLC-V-0206, the PLC system tasks (communication,...) can also be assigned to the various cores.

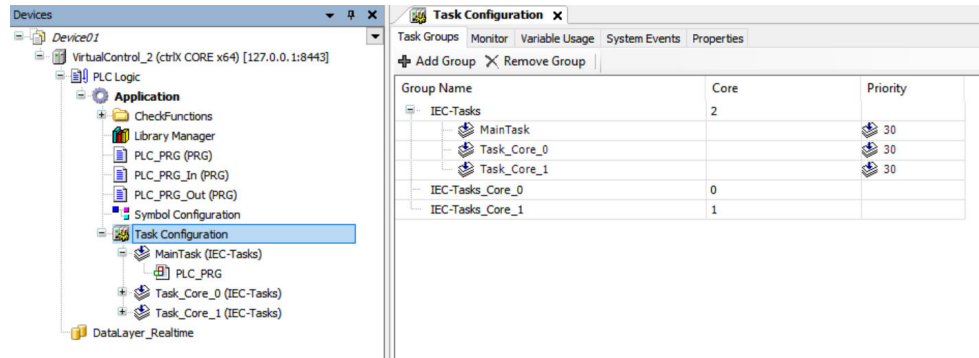


Fig. 7: ctrlX PLC Engineering task configuration

Requirements

- The ctrlX CORE control supports multicore functionality (from device description V1.10.0.1)
- You have defined at least two tasks in your application, for example "Main-Task (IEC task)" and "LowTask (IEC task)"

Configuring the assignment

The tasks are assigned to the CPU cores via ctrlX PLC Engineering in the Task Groups tab, see [further documentation](#).

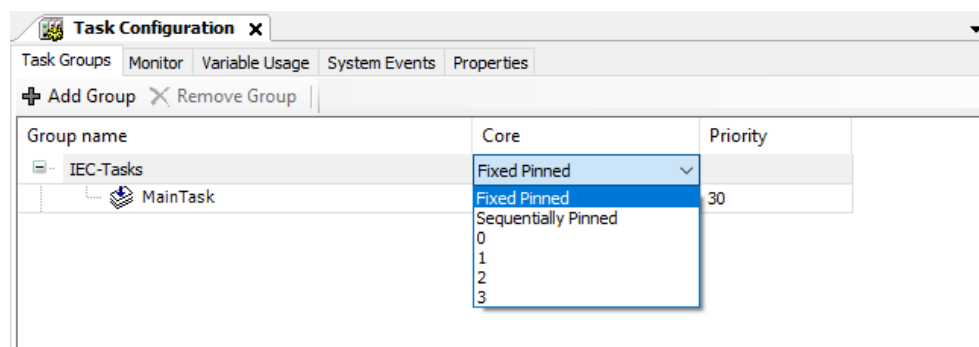


Fig. 8: Task groups tab

In the as delivered condition, the "IEC tasks" task group is already preconfigured in the "Fixed Pinned" setting, in which all tasks in CPU core #2 are processed.

To divide tasks between different cores, first create another task group. Then select the CPU number that is to be responsible for processing the second task group in the "Core" field. Now drag & drop the desired tasks into the new task group.

In the "Sequentially pinned" setting, the CPU cores are assigned sequentially



Please note that the ctrlX OS operating system is running on CPU cores 0 and 1.

If task groups are running on these cores, the watchdog of the IEC task should always be activated.

Functional description

The system defines four (system) task groups to subdivide the functionality:

- **System**

All tasks of the system, such as scheduler, exception handler, etc.

- **Communication**

All communication tasks.

- **Visualization**

All tasks edit the visualization.

- **Asynchronous operations**

All tasks of the asynchronous operations e.g: SysFileAsync

These task groups and their configuration are mapped in the Data Layer on the node: plc->admin->runtime-settings->SysCpuMultiCore'

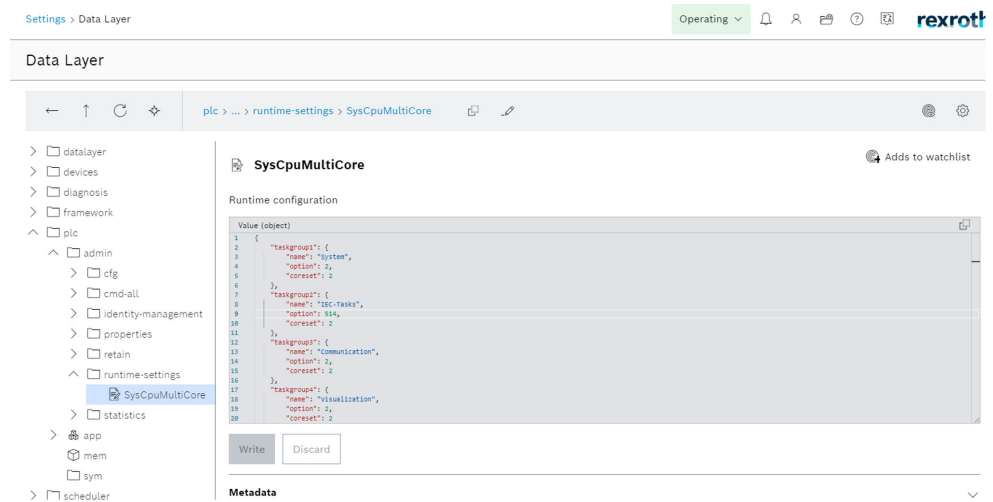


Fig. 9: ctrlX OS – Data Layer

The name and options of the respective task group are fixed and cannot be changed. The “coreset” parameter can be used to select the processor core on which this task group is to be processed. The processor cores available on the ctrlX OS control for this assignment are also mapped under this Data Layer node:

SysCpuMultiCore

Runtime configuration

	Value (object)
22	"taskgroup5": {
23	"name": "Asynchronous Operations",
24	"option": 2,
25	"coreset": 2
26	},
27	"ieccoreset": [
28	0,
29	1,
30	2,
31	3
32	]
33	}
34	

Fig. 10: ctrlX OS – Data Layer / available processor cores



IEC tasks always have a higher priority than the operating system and can slow down the operating system considerably or render it unusable if the load is too high.

Notes

- Changes to the standard configuration affect the runtime behavior of the overall system and are only recommended for experts.
- The ctrlX OS control has to be restarted for the new settings for the task group(s) to take effect.

Further information on the multicore application can be found in the PLC Engineering documentation, see [Multicore and task configuration](#)

8 Flexible configuration of the memory for remanent PLC data



The memory for remanent PLC data is available as a pool for all PLC applications that are operated on the control during runtime.

Remanent variables have the property of retaining their values beyond a control switch-off process.

Physically, up to 128 kB of non-volatile memory is available on the ctrlX CORE control. The memory space can be requested by the apps and used for remanent variables. For the PLC app and the PLC applications, the default value for variables of type VAR RETAIN and VAR PERSISTENT is 64 kB.

From PLC App version PLC-V-0108, the reserved memory size can be set dynamically to up to 120 kB.



If the memory size is changed, the control has to be rebooted.
Existing retain and retain persistent data is lost.

Configuring the memory size

1. ➤ Open the ctrlX OS web interface and navigate to the “PLC” window, see [➔ Documentation](#)
2. ➤ In the header, click on 
 - ➔ The dialog “Allocate PLC retain memory (Bytes)” opens
3. ➤ Enter the desired memory size in bytes.
The possible maximum value depends on the hardware. If the input value is outside the minimum or maximum memory size, the minimum or maximum value is entered. If you cancel the process, the last valid value is retained.
4. ➤ Confirm the dialog
 - ➔ The control unit performs a reboot

The change to the memory size is now complete

9 Connecting the PLC runtime system to the ctrlX OS system behavior

Connection to the ctrlX OS System Handler

The PLC applications and the associated tasks are managed via the scheduler of the PLC runtime system.

To ensure the general system behavior of the ctrlX CORE control, a connection to the ctrlX OS system status has also been established. For the PLC app:

- The PLC applications are taken into account in the ctrlX OS system status as follows:
 - Each PLC application is registered with the System Handler of the ctrlX OS Runtime.
 - The system status (Data Layer node system->state) shows the PLC applications currently loaded on the device.
 - The IEC tasks generated via the PLC application are recorded in the task overview of the ctrlX OS Scheduler, but the task properties cannot be changed in the associated dialog.
- When switching the operating state of the ctrlX CORE control, the PLC applications are handled as follows:
 - **Operating:** All PLC applications on the device are set to the "RUN" state.
 - **Setup:** All PLC applications currently running on the device are switched to the "STOP" state.
 - **Service:** All PLC applications currently running on the device are switched to the "STOP" state.
- Switching the status of a PLC application:
 - **Operating:** Each PLC application can switch from RUN to STOP and vice versa as required.
 - **Setup:** All PLC applications configured on the ctrlX device are initially in the **STOP** state and can be switched to the RUN state. As soon as at least one PLC application has been started, the scheduler status changes to "Partial Operating".
 - **Service:** All PLC applications loaded onto the device are in the "STOP" state and cannot be switched to "RUN".

Assigning PLC tasks depending on the control architecture

The standard architecture for ctrlX CORE controls is designed for 4 processor cores (core0,..core3). A distinction is made between real-time and non-real-time processor cores. The real-time cores core 2 and core 3 are reserved

for real-time applications, while the standard applications and the Linux operating system are executed on core 0 and 1. The first real-time core (core 2) is reserved for the execution of the PLC runtime environment and the PLC applications.

In case of control architectures that deviate from the standard architecture, the System Handler determines the processor cores available on the hardware and assigns the properties realtime or non-realtime. The first real-time core is always assigned to the PLC runtime system.



In ctrlX PLC Engineering, 4 cores are always provided to assign task groups via the task configuration, regardless of the target system architecture. If the configured cores are not available on the target architecture, the download process is aborted with the corresponding information.

10 Optimized access to complex IEC data types via the Data Layer

There are several ways to access variables of a PLC program in the Data Layer. However, structures and arrays of structures have previously only been mapped with the original structure in the data layer. Version PLC-V-0204 introduces a new function that enables new data types for selected variables to be created in the Data Layer in the symbol configuration.

Prerequisite

To be able to use the new function, the following options have to be activated in ctrlX PLC Engineering in the symbol configuration settings:

- Support OPC UA features
- Support calls to Functions, FBs, Methods and Programs

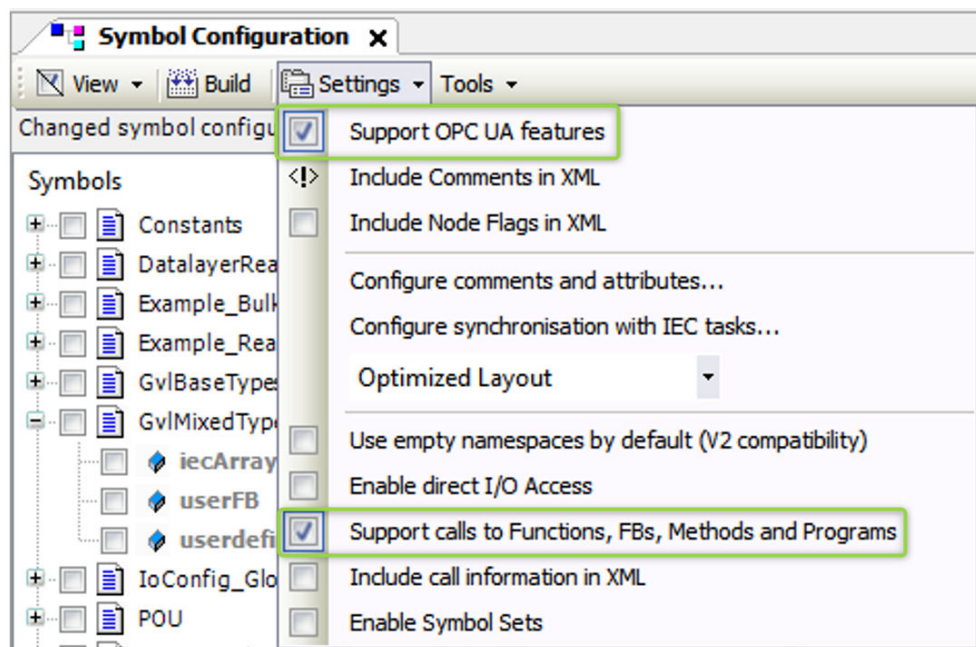


Fig. 11: Symbol configuration settings in ctrlX PLC Engineering

10.1 Using structures

To facilitate a more user-friendly access to complex data types, the data type also has to be made known to the Data Layer. If a new structure is created in the IEC program, the program can simply be downloaded with this structure.

Example: User-defined structure with 3 elements

Declaration of the user-defined structure in PLC Engineering:

```

1  TYPE UserdefinedStruct :
2  STRUCT
3      structtest : BOOL;
4      structtest2 : BYTE;
5      structtest3 : INT;
6  END_STRUCT
7  END_TYPE

```

Following the program download to the ctrlX device, a new data type with a flat buffer schema in the ctrlX OS web interface is displayed in the Data Layer under the "types" node:

- types
 - apps
 - common
 - datalayer 65
 - diagnosis
 - health
 - plc
 - app
 - ApplicationArray
 - UserdefinedStruct
 - app-date-and-time
 - app-idm-user-infos
 - app-idm-user-login
 - app-info
 - app-load-applications
 - app-memory-area
 - app-memory-area-access

UserdefinedStruct

Type information

```

1  {
2    "$schema": "https://json-schema.org/draft/2019-09/schema",
3    "definitions": {
4      "UserdefinedStruct": {
5        "type": "object",
6        "properties": {
7          "structtest": {
8            "type": "boolean",
9            "default": false
10         },
11         "structtest2": {
12           "type": "integer",
13           "minimum": 0,
14           "maximum": 255,
15           "default": 0
16         }
17       }
18     }
19   }

```

Metadata

Name	Value
Node Class	Type
Operations	read browse
Display format	Auto
References	readType: types/datalayer/reflection

JSON Flatbuffer-Schema:

```

{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "UserdefinedStruct": {
      "type": "object",
      "properties": {
        "structtest": {
          "type": "boolean",
          "default": false
        },
        "structtest2": {
          "type": "integer",
          "minimum": 0,
          "maximum": 255,
          "default": 0
        }
      }
    }
  }
}

```

```

        "structtest3": {
          "type": "integer",
          "minimum": -32768,
          "maximum": 32767,
          "default": 0
        },
        "additionalProperties": false
      },
    },
    "$ref": "#/definitions/UserdefinedStruct"
  }

```

The new registered data type in the Data Layer is linked in the metadata under „References“ for the regular representation of the structure variable with the help of child nodes for the individual elements

userdefinedStruct

Value (object)

```

1 {
2   "structtest": true,
3   "structtest2": 15,
4   "structtest3": -1913
5 }
6

```

Write Discard

Child nodes

Metadata

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto
References	readType: types/plc/app/ApplicationArray/UserdefinedStruct writeType: types/plc/app/ApplicationArray/UserdefinedStruct

Consequently, the IEC program structure is displayed in full in the „Value“ area.

Depending on the access rights read/write, these values can now be edited and changed in the IEC program using a write command.

10.2 Using arrays of structures

If an array is created from a structure in the program, the representation is similar.

Example: Array of a user-defined structure

```
UserArray: ARRAY[0..15] OF UserdefinedStruct :=
```

For display in the Data Layer of the ctrlX OS web interface, a type is automatically generated that places the wording „ArrayOf“ in front of the user-defined structure:

The screenshot shows the 'ArrayOfUserdefinedStruct' type information in the ctrlX OS web interface. On the left, a tree view shows the hierarchy: types > apps > common > datalayer > diagnosis > health > plc > app > ApplicationArray > ArrayOfUserdefinedStruct. The main panel displays the type information and metadata.

Type information

```
Value (object)
1 {
2   "$schema": "https://json-schema.org/draft/2019-09/schema",
3   "definitions": {
4     "ArrayOfUserdefinedStruct": {
5       "type": "object",
6       "properties": {
7         "value": {
8           "type": "array",
9           "items": {
10            "$ref": "#/definitions/UserdefinedStruct"
11          }
12        }
13      },
14      "additionalProperties": false
15    },
16    "UserdefinedStruct": {
```

Metadata

Name	Value
Node Class	Type
Operations	read browse
Display format	Auto
References	readType: types/datalayer/reflection

JSON Flatbuffer-Schema:

```
{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "ArrayOfUserdefinedStruct": {
      "type": "object",
      "properties": {
        "value": {
          "type": "array",
          "items": {
            "$ref": "#/definitions/
UserdefinedStruct"
          }
        }
      },
      "additionalProperties": false
    },
    "UserdefinedStruct": {
      "type": "object",
      "properties": {
        "structtest": {
          "type": "boolean",
          "default": false
        },
        "structtest2": {
```

```

        "type": "integer",
        "minimum": 0,
        "maximum": 255,
        "default": 0
    },
    "structtest3": {
        "type": "integer",
        "minimum": -32768,
        "maximum": 32767,
        "default": 0
    }
},
"additionalProperties": false
}
},
"$ref": "#/definitions/ArrayOfUserdefinedStruct"
}

```

As well as for user-defined structures, the same applies to the use of arrays:

Example: ctrlX OS web interface/Data Layer

The screenshot shows the 'UserArray' configuration in the ctrlX OS web interface. The left sidebar displays a tree view with 'UserArray' selected under 'GvlMixedTypes'. The main panel shows the 'UserArray' configuration, including a JSON schema for the array, 'Write' and 'Discard' buttons, and a metadata table.

Value (object)

```

1 {
2   "value": [
3     {
4       "structtest": true,
5       "structtest2": 1,
6       "structtest3": 11
7     },
8     {
9       "structtest": true,
10      "structtest2": 2,
11      "structtest3": 22
12    },
13    {
14      "structtest": true,
15      "structtest2": 3,
16      "structtest3": 33
17    }
18  ]
19 }

```

Child nodes

Metadata

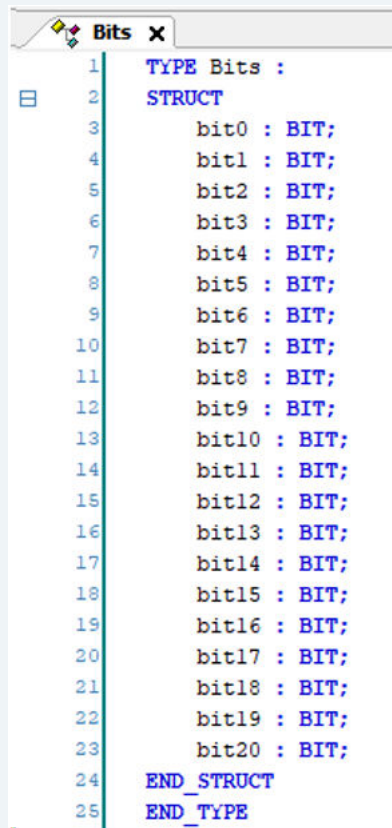
Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto
References	readType: types/plc/app/ApplicationArray/ArrayOfUserdefinedStruct writeType: types/plc/app/ApplicationArray/ArrayOfUserdefinedStruct

The newly created flat buffer schema is stored as a reference and the variable can be displayed completely in the Data Layer.

10.3 Using structures with bits

From version PLC-V-0206, the use of a data type bit within a structure is supported.

Example: Declaration in ctrlX PLC Engineering



```

1  TYPE Bits :
2  STRUCT
3      bit0 : BIT;
4      bit1 : BIT;
5      bit2 : BIT;
6      bit3 : BIT;
7      bit4 : BIT;
8      bit5 : BIT;
9      bit6 : BIT;
10     bit7 : BIT;
11     bit8 : BIT;
12     bit9 : BIT;
13     bit10 : BIT;
14     bit11 : BIT;
15     bit12 : BIT;
16     bit13 : BIT;
17     bit14 : BIT;
18     bit15 : BIT;
19     bit16 : BIT;
20     bit17 : BIT;
21     bit18 : BIT;
22     bit19 : BIT;
23     bit20 : BIT;
24 END_STRUCT
25 END_TYPE
    
```

These bits are registered in the Data Layer under the „types“ node with a flat buffer scheme as boolean variable:

Example: ctrlX OS web interface/Data Layer

- ^ types
 - > apps
 - > common
 - > datalayer 65
 - > diagnosis
 - > health
 - ^ plc
 - ^ app
 - ApplicationArray
 - Bits**
 - app-date-and-time
 - app-idm-user-infos
 - app-idm-user-login
 - app-info
 - app-load-applications
 - app-memory-area
 - app-memory-area-access
 - app-operation-state
 - app-project-info
 - app-reset
 - app-state
 - app-task-info
 - app-tasks-info

Bits

Type information

Value (object)

```

1 {
2   "$schema": "https://json-schema.org/draft/2019-09/schema",
3   "definitions": {
4     "Bits": {
5       "type": "object",
6       "properties": {
7         "bit0": {
8           "type": "boolean",
9           "default": false
10        },
11        "bit1": {
12          "type": "boolean",
13          "default": false
14        },
15        "bit2": {
16          "type": "boolean",

```

Metadata

Name	Value
Node Class	Type
Operations	read browse
Display format	Auto
References	readType: types/datalayer/reflection

Individual bits are also represented as nodes of type Bool.

JSON flat buffer scheme:

```

{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "Bits": {
      "type": "object",
      "properties": {
        "bit0": {
          "type": "boolean",
          "default": false
        },
        "bit1": {
          "type": "boolean",
          "default": false
        },
        "bit2": {
          "type": "boolean",
          "default": false
        },
        "bit3": {
          "type": "boolean",
          "default": false
        },
        "bit4": {
          "type": "boolean",
          "default": false
        }
      }
    }
  }
}

```

```

"bit5": {
  "type": "boolean",
  "default": false
},
"bit6": {
  "type": "boolean",
  "default": false
},
"bit7": {
  "type": "boolean",
  "default": false
},
"bit8": {
  "type": "boolean",
  "default": false
},
"bit9": {
  "type": "boolean",
  "default": false
},
"bit10": {
  "type": "boolean",
  "default": false
},
"bit11": {
  "type": "boolean",
  "default": false
},
"bit12": {
  "type": "boolean",
  "default": false
},
"bit13": {
  "type": "boolean",
  "default": false
},
"bit14": {
  "type": "boolean",
  "default": false
},
"bit15": {
  "type": "boolean",
  "default": false
},
"bit16": {
  "type": "boolean",
  "default": false
},
"bit17": {
  "type": "boolean",
  "default": false
},
"bit18": {
  "type": "boolean",
  "default": false
},
"bit19": {

```

```

        "type": "boolean",
        "default": false
    },
    "bit20": {
        "type": "boolean",
        "default": false
    }
},
"additionalProperties": false
}
},
"$ref": "#/definitions/Bits"
}

```

With the reference of the flat buffer scheme, the entire structure of the bits can now be adjusted without querying each bit individually.

Example: ctrlX OS web interface/Data Layer

The screenshot shows the ctrlX OS web interface. On the left is a tree view of the Data Layer structure. The 'bitsTest' variable is selected under 'types/plc/app/ApplicationUserdefinedSymbols/Bits'. On the right, the 'bitsTest' variable is displayed with its value (an object) and metadata.

bitsTest

Value (object)

```

1 {
2   "bit0": false,
3   "bit1": true,
4   "bit2": false,
5   "bit3": true,
6   "bit4": false,
7   "bit5": true,
8   "bit6": false,
9   "bit7": true,
10  "bit8": false,
11  "bit9": true,
12  "bit10": false,
13  "bit11": true,
14  "bit12": false,
15  "bit13": true,
16  "bit14": false,

```

Write Discard

Child nodes

Metadata

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto
References	readType: types/plc/app/ApplicationUserdefinedSymbols/Bits writeType: types/plc/app/ApplicationUserdefinedSymbols/Bits

10.4 Using enums

Enums have previously been treated as simple integer variables.

From version PLC-V-0206, flat buffer schemes are also created for enums so that they can be used with their own designations, as shown in the following example.

Example: Declaration enum test variable in ctrlX PLC Engineering

```
EnumTest X
1  TYPE EnumTest :
2  (
3      enumTestMember1 := 0,
4      enumTestMember2 := 1,
5      enumTestMember3 := 2
6  );
7  END_TYPE
8
```

This enum is fully described in a flat buffer schema under the "types" node:

Example: ctrlX OS web interface/Data Layer

types

apps

common

datalayer657

diagnosis

health

plc

app

ApplicationUserdefinedSymbols

EnumTest

app-date-and-time

app-idm-user-infos

app-idm-user-login

app-info

app-load-applications

app-memory-area

app-memory-area-access

app-operation-state

app-project-info

app-reset

app-state

app-task-info

app-tasks-info

EnumTest

Type information

Value (object)

1 {

2 "\$schema": "https://json-schema.org/draft/2019-09/schema",

3 "definitions": {

4 "EnumTest": {

5 "type": "string",

6 "enum": [

7 "enumTestMember1",

8 "enumTestMember2",

9 "enumTestMember3"

10]

11 },

12 "": {

13 "type": "object",

14 "properties": {

15 "EnumTest": {

16 "\$ref": "#/definitions/EnumTest",

Metadata

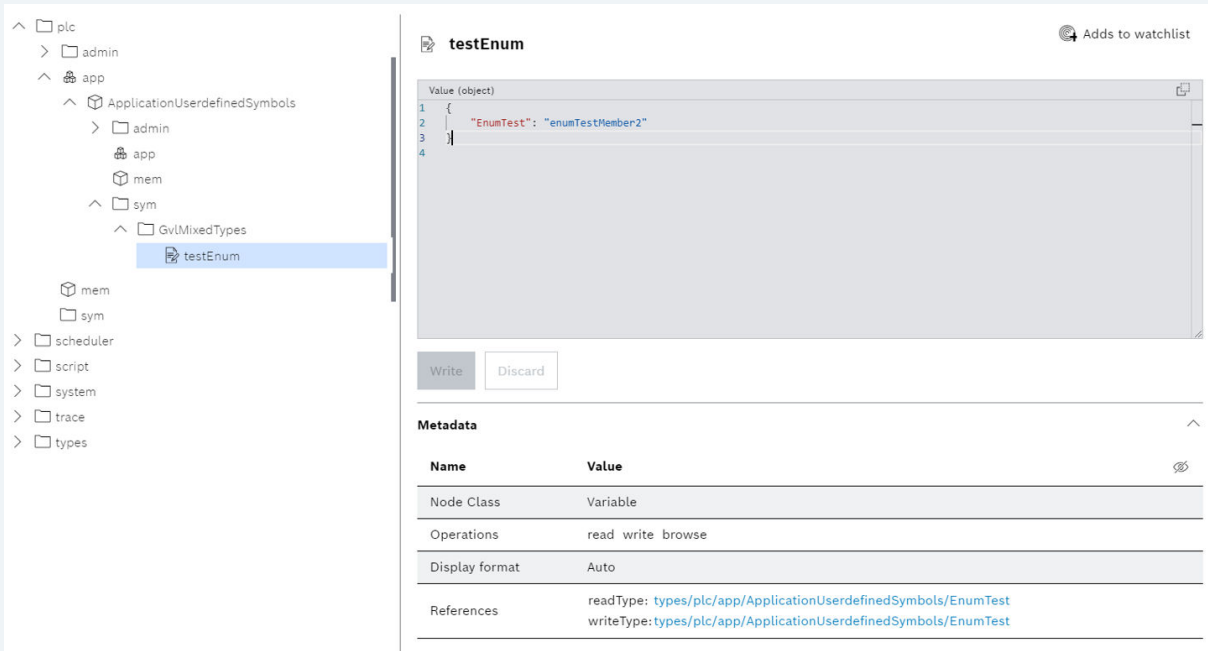
Name	Value
Node Class	Type
Operations	read browse
Display format	Auto
References	readType:types/datalayer/reflection

JSON flat buffer scheme:

```
{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "EnumTest": {
      "type": "string",
      "enum": [
        "enumTestMember1",
        "enumTestMember2",
        "enumTestMember3"
      ]
    },
    "": {
      "type": "object",
      "properties": {
        "EnumTest": {
          "$ref": "#/definitions/EnumTest",
          "default": "enumTestMember1"
        }
      },
      "additionalProperties": false
    }
  },
  "$ref": "#/definitions/"
}
```

The enum is now displayed in the node of the Data Layer as shown in the following example:

Example: ctrlX OS web interface/Data Layer



The screenshot shows the ctrlX OS web interface. On the left, a tree view displays the project structure, with 'testEnum' selected under 'GvlMixedTypes'. The main area shows the 'testEnum' node with a value of 'enumTestMember2'. Below the value, there are 'Write' and 'Discard' buttons. A 'Metadata' section is expanded, showing a table with the following data:

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto
References	readType: types/plc/app/ApplicationUserdefinedSymbols/EnumTest writeType: types/plc/app/ApplicationUserdefinedSymbols/EnumTest

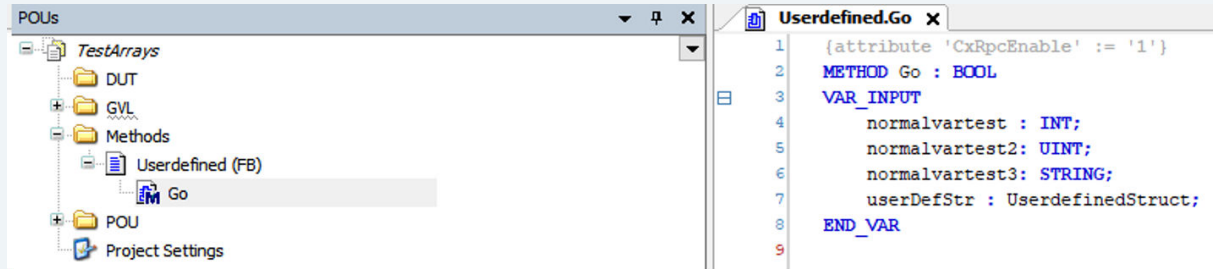
An integer value is no longer displayed, instead it is now possible to use the name directly.

An integer value can also be assigned, but this is displayed as an enum after the „Write“ statement.

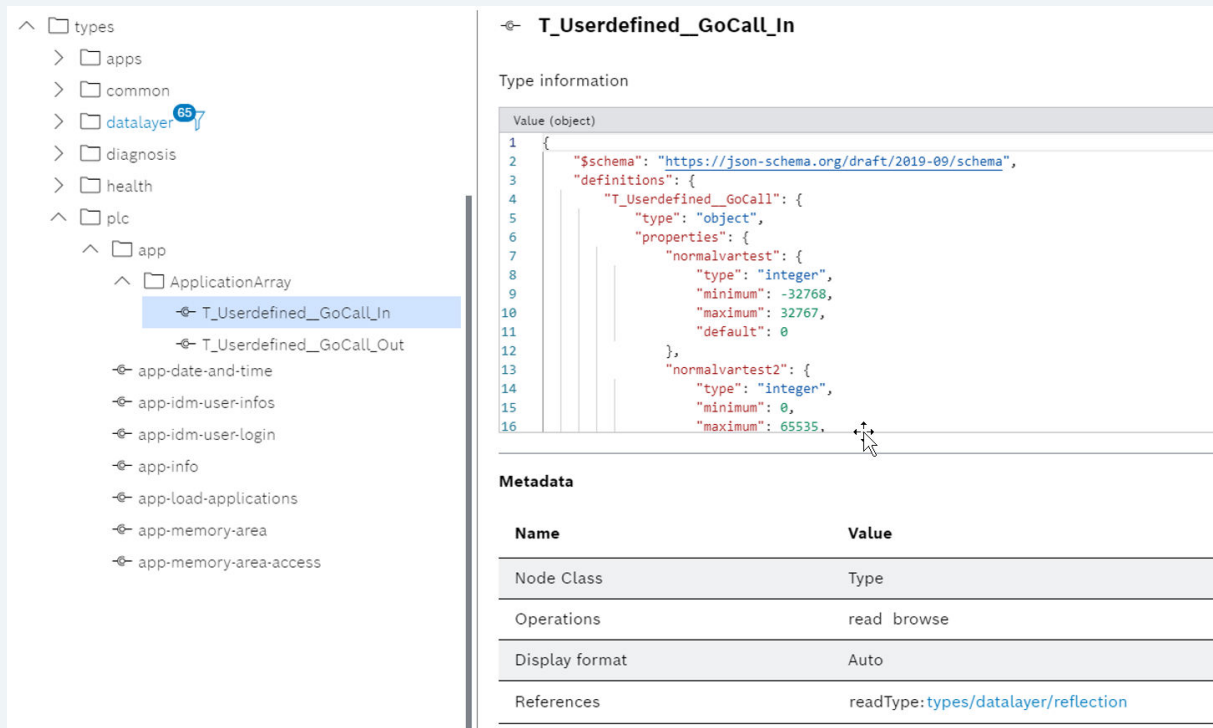
10.5 Method calls

If a method with parameters is created, a new data type is also created for this method in the Data Layer.

Example: Declaration in PLC Engineering



As there are both input and output variables, a separate data type is created for each variable type in the Data Layer of the ctrlX OS web interface:



JSON Flatbuffer-Schema:

```

{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "T_Userdefined_GoCall": {
      "type": "object",
      "properties": {
        "normalvartest": {
          "type": "integer",
          "minimum": -32768,
          "maximum": 32767,
          "default": 0
        },
        "normalvartest2": {

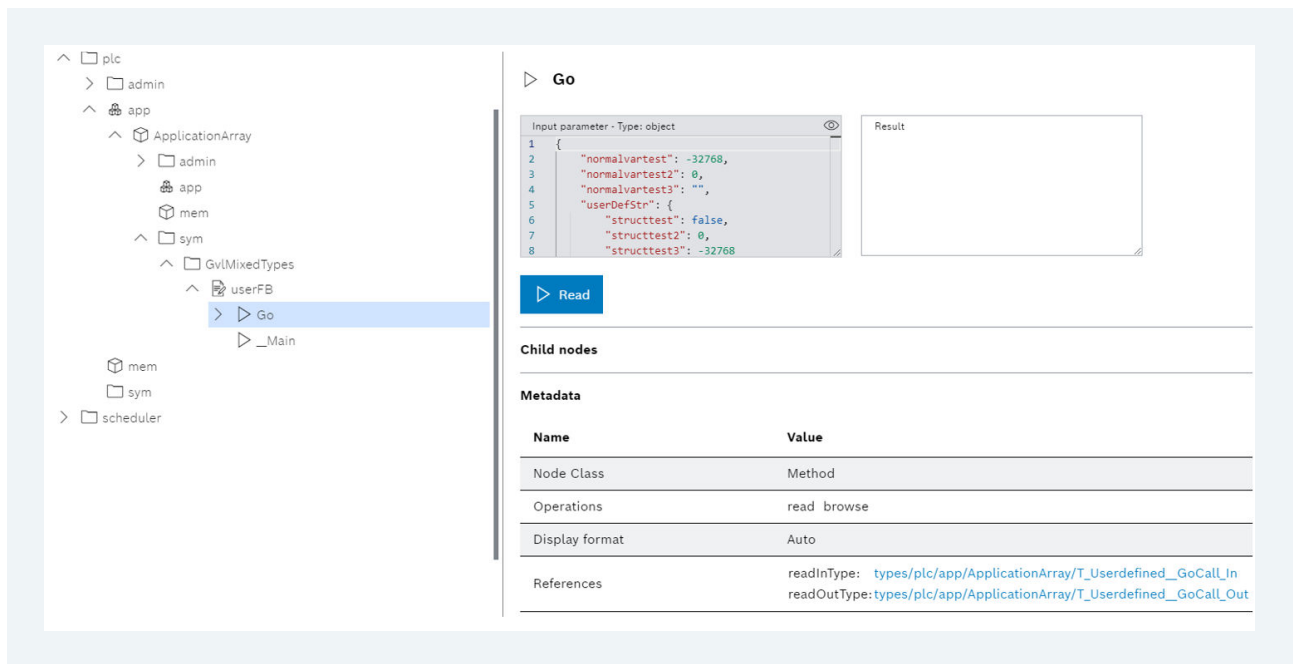
```

```

        "type": "integer",
        "minimum": 0,
        "maximum": 65535,
        "default": 0
    },
    "normalvartest3": {
        "type": "string"
    },
    "userDefStr": {
        "$ref": "#/definitions/UserdefinedStruct"
    }
},
"required": [
    "normalvartest3",
    "userDefStr"
],
"additionalProperties": false
},
"UserdefinedStruct": {
    "type": "object",
    "properties": {
        "structttest": {
            "type": "boolean",
            "default": false
        },
        "structttest2": {
            "type": "integer",
            "minimum": 0,
            "maximum": 255,
            "default": 0
        },
        "structttest3": {
            "type": "integer",
            "minimum": -32768,
            "maximum": 32767,
            "default": 0
        }
    }
},
"additionalProperties": false
}
},
"$ref": "#/definitions/T_Userdefined__GoCall"
}

```

This allows the method to be called directly with all parameters, as in the following example in the Data Layer of the ctrlX OS web interface:



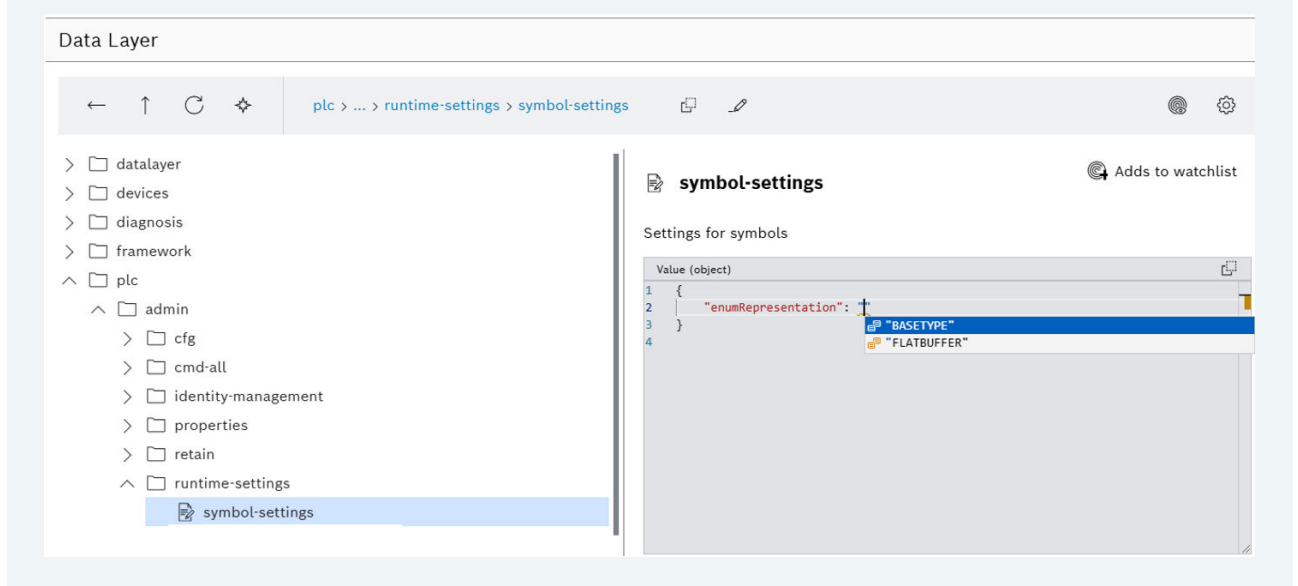
10.6 Setting options

10.6.1 Enum representation

It is possible to change the representation of ENUM data types.

Up to now, variables of type ENUM have always been displayed with their basic types as specified in the IEC program. During the development process, this representation was changed, particularly with regard to OPC UA. By default, each ENUM data type is now stored and displayed as a flat buffer. This allows you to work with the element names. In order to remain downward compatible, it is possible to continue using the old display variant. The following entry can now be found under the „plc/admin/runtime-settings/symbol-settings“ path:

Data Layer: symbol-settings in the Data Layer of the ctrlX OS web interface



Possible values for „enumRepresentation“:

- **BASETYPE**: ENUM basic data types are used for representation.
- **FLATBUFFER** (Default): Flat buffer schmes are used for ENUM data type representation.

For example, an IEC type DINT with the setting “BASETYPE” is represented as follows:

Example: ctrlX OS web interface/Data Layer

The screenshot shows the 'Data Layer' interface for the variable `iecEnumColorDint`. The breadcrumb path is `plc > ... > GvlMixedTypes > iecEnumColorDint`. The variable is selected in the left sidebar. The main panel displays the variable's value as `Value (int32)*` with the value `2`. Below the value field are 'Write' and 'Discard' buttons. The 'Metadata' section shows the following table:

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto

With the „FLATBUFFER“ option, the variable is contained in the Data Layer as follows:

Example: ctrlX OS web interface/Data Layer

The screenshot shows the 'Data Layer' interface for the variable `iecEnumColorDint`. The breadcrumb path is `plc > ... > GvlMixedTypes > iecEnumColorDint`. The variable is selected in the left sidebar. The main panel displays the variable's value as `Value (object)` with a flatbuffer JSON structure:

```

1 {
2   "EnumColorDINT": "blue"
3 }
4

```

Below the value field are 'Write' and 'Discard' buttons. The 'Metadata' section shows the following table:

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto

10.6.2 Browsing array variables via OPC UA

From version 2.6, the "display name" element is used in the meta data for browsing array variables via OPC UA. In previous version, the browsing name has always been evaluated as display name. This can, for example, lead to incorrect displays in HMI applications that were created on this basis and reference OPC UA array variables in the PLC project.

To avoid having to change such an existing application, it is possible to configure the display via the „plc/admin/runtime-settings/symbol-settings“ path, using the „ArrayDisplayname“ parameter.

Possible values for „ArrayDisplayname“:

- **ON** (default): Publish display names via metadata (default).
- **OFF** : Do not specify display names (mapping of array variables compatible with versions <PLC-V-0206).

10.7 Restrictions

If structures or arrays are declared for which no flatbuffer schema can be created, no new type is added under the data types. In this case, error messages are displayed in the logbook of the ctrlX device.

The following restrictions apply at the time of issue of the documentation:

- Structures with bits must not be mixed with other types. These structures can only be used as mere bit structures.
- Method calls with structures including bits are not supported.
- Pointers in structures or arrays are not supported (references are possible).

11 Task configuration

11.1 General information on task configuration

The PLC user tasks are managed via the scheduler of the PLC runtime system. The scheduler of the PLC runtime system is not to be confused with the ctrlX OS scheduler and is clocked with a fixed interval of one millisecond. At this interval, the scheduler cyclically checks which PLC tasks of type "Cyclic", "Event", "Free-wheeling" and "Status" are enabled for processing. The cyclic tasks can therefore only be configured in the task configuration with a minimum cycle time and as a multiple of one millisecond.

This restriction also applies to the configuration and handling of times for the task watchdog. As the PLC scheduler also cyclically checks the runtimes of all currently running PLC tasks, including the event-controlled tasks, for exceeding the configured watchdog times in the specified cycle, only values in multiples of one millisecond can be assigned.

If cycle times of less than one millisecond are required, this can be implemented using the "External" task type. The scheduler of the ctrlX OS system also allows the configuration of tasks with cycle times of less than one millisecond, in the context of which an event is generated which in turn signals the start of the assigned PLC task.

Note: The starting time and runtime behavior of a PLC task depends on the configured priority. Even if the PLC scheduler has enabled the processing of a task, it can only run as long as no other task in the system with a "higher" priority is active on the assigned processor core.

11.2 Event sources for external event-controlled PLC tasks

Der ctrlX scheduler contains two types of program configuration:

- Automatic program configuration (new from version PLC-V-0206).
- Freely configurable, external event sources.

Additional information on scheduling & timing, see: [ctrlX OS Documentation](#)

11.3 Support of the automatic program configuration of the ctrlX scheduler

From version PLC-V-0206, the PLC app supports the automatic program configuration of the ctrlX scheduler.

Using external events in ctrlX PLC Engineering for task control

In the configuration dialog of a PLC task, ctrlX PLC Engineering provides the following new external events for the task type "External":

External event	Description
AFTER_CTRLX_ETHERCAT_MASTER	This event triggers a PLC task after the ctrlX EtherCAT master has provided the current output values.
AFTER_CTRLX_MOTION_VALUE_GENERATION	This event triggers a PLC task after the ctrlX Motion has provided the current output values.
AFTER_CTRLX_PROFINET_DEVICE	This event triggers a PLC task after the ctrlX Profinet Device has provided the current output values.

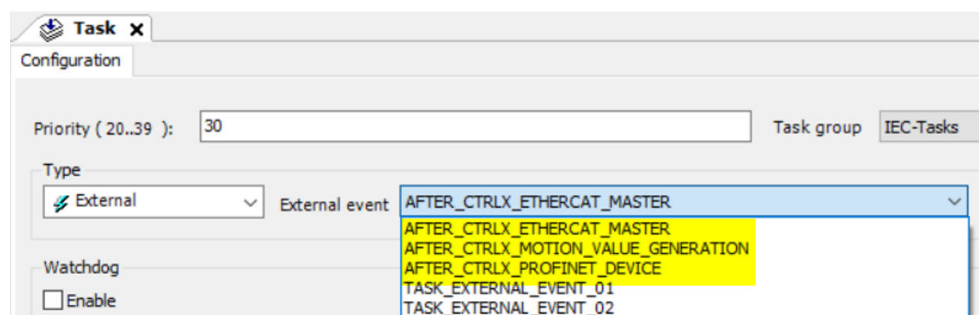


Fig. 12: External event in ctrlX PLC Engineering

If the ctrlX scheduler is in mode "Automatic program configuration" and if a PLC program containing tasks using one of the new external events is downloaded, a "Callable" is automatically created, triggering the PLC task. This means that the user does not have to configure anything in the scheduler.

Example: Triggering a PLC task directly after the Motion value has been calculated

1. Create an external event-controlled task in the PLC program that is linked to the external event „AFTER_CTRLX_MOTION_VALUE_GENERATION“.

Info: The Motion value is calculated in the ctrlX scheduler task „ctrlXAutomation“ as „callable“.

2. Load the PLC program to the control.

➔ When downloading the PLC program, a „callable“ is automatically created for the PLC in the scheduler, running after the Motion “callable” and triggering the PLC task.

The following “callables” are created in a scheduler task if a corresponding external event is used in the PLC program:

Name of the „callables“	Scheduler task
plc-after-ctrlx-ethercat-master	ctrlXAutomation
plc-after-ctrlx-motion-value-generation	ctrlXAutomation
plc-after-ctrlx-profinet-device	profinetdevice

The “callables” use the following synchronization points to establish the order of all “callables” in a scheduler task:

Name of the „callables“	“Before” synchronization points	“After” synchronization points
plc-after-ctrlx-ethercat-master	axisprofile-adaption-entry motion-entry oscilloscope-entry	fieldbuses-ethercat-master-exit
plc-after-ctrlx-motion-value-generation	axisprofile-distribution-entry oscilloscope-entry	motion-value-generation-exit
plc-after-ctrlx-profinet-device		fieldbuses-profinet-device-exit

11.4 Freely configurable, external event sources

ctrlX PLC Engineering provides several types for selection in the configuration dialog of a PLC task. The task type "External" provides the external events TASK_EXTERNAL_EVENT_01 to TASK_EXTERNAL_EVENT_10. These events are initially not assigned on the control. The following action instruction describes how to configure external events.



Up to ctrlX PLC App version PLC-V-0114, TASK_EXTERNAL_EVENT_07 was created automatically.

As of ctrlX PLC App version PLC-V-0116, events have to be configured as described below.

Configuring an external event

1. Switch the ctrlX device to the "Setup" operating state using the status buttons in the header
2. Use the side navigation to call the “Scheduler” window.
3. In the window, select a task in the context the event is to be generated in.

If you want to synchronize your PLC task with the Motion and the EtherCAT communication, please select the task "ctrlXAutomation".

Optionally, create a new Task with the desired properties.

4. ➤ Use the button  to open the list of callables that are called up in the associated task context.
5. ➤ Use the button  to open the dialog for adding a callable and select **plc-external-event** with the desired Run-Index.
 - ➔ The dialog for selecting the event is displayed and offers the arguments **TASK_EXTERNAL_EVENT_01** to **TASK_EXTERNAL_EVENT_10** for selection.
6. ➤ Select the event and delete all other entries so that only one argument is displayed.

The selection of multiple arguments or the multiple use of an argument in different tasks is not possible.
7. ➤ Switch the ctrlX device back to the operating state "Operating".
 - ➔ The events are generated now.
8. ➤ Finally, assign the previously configured event to the desired PLC task in the task configuration in ctrlX PLC Engineering.
9. ➤ Load your program to the controller and check if the PLC task is called.

Use case

A typical use case is, for example, the synchronization of a PLC task with the EtherCAT master.

Synchronization is achieved by extending the "ctrlXAutomation" task by a Callable of type "plc-external-event" with a higher run index. This means that the EtherCAT master is processed first and then the external event for processing the PLC task is generated.

12 Creating external POU's

In addition to creating POU's (program organization units) in the input languages of IEC 6-1131, they can also be implemented "externally". The difference to program-internal POU's is that although the POU interfaces are defined in IEC as usual, the POU's are implemented in C or C++ in the PLC runtime system.

If external POU's are used in the program, the associated "linking" only takes place during the download to the target system. If external POU's cannot be resolved, the PLC runtime system reports the error „Unresolved external ...“.

For further information on the creation of externally implemented POU's, see: [➔ Web documentation](#)



To create C/C++ functions for ctrlX OS controls, please use the ctrlX OS SDK (Software Development Kit), see the following web links:

[➔ ctrlX SDK How-To](#)

[➔ ctrlX SDK on GitHub](#)

13 ctrlX UI – Elements

13.1 Navigation

13.1.1 Side navigation – PLC App

With the installation of the PLC App, the following entries are added to the web interface of the ctrlX device:

Side navigation – Window PLC

The “PLC” window is used to manage the PLC applications on the ctrlX device, see:

➔ [Window – “PLC”](#)

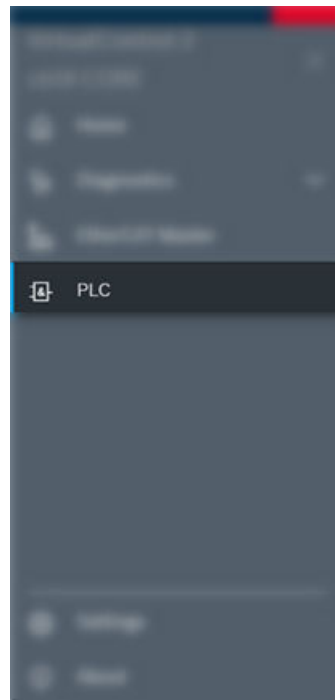


Fig. 13: Side navigation - “PLC”

Widget in the start window of the web interface

The Widget “PLC” displays information on the operating status of the configured PLC applications and provides direct calls to the following areas:

- “PLC” window
- Software tool ctrlX PLC Engineering for creating the PLC project

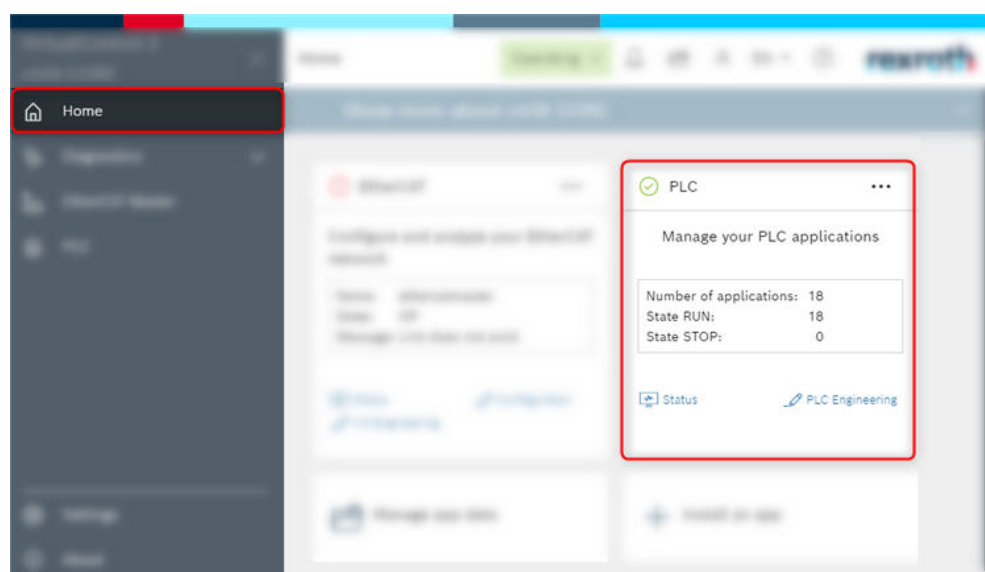


Fig. 14: “PLC” widget

13.2 Windows

13.2.1 Window – “PLC”

The “PLC” window is used to manage the PLC applications on the ctrlX device.

The header at the top of the window can be used to centrally control the operating status of all configured applications and reset them if necessary, see:

➔ [Header – “PLC”](#)

Further control options and information are provided below the header in the following tabs:

- **Tab “Applications”**
Shows all applications that are configured or loaded on the ctrlX device and provides buttons for commanding individual applications (Start / Stop / Reset), see:
➔ [Tab – “Applications”](#)
- **Tab “Boot applications”**
Provides a tabular list of all boot applications configured on the ctrlX device and their creation time, see:
➔ [Tab – “Boot applications”](#)
- **Tab “Project information”**
Provides general information about the PLC project in a comparative view (target application vs. source application), see:
➔ [Tab – “Project information”](#)

Call

In the ctrlX OS web interface:

“side navigation → PLC”

Configuring the application

Immediately after installing the PLC App, no PLC application has yet been configured on the ctrlX device.

A PLC application is configured in the software tool ctrlX PLC Engineering which is part of the ctrlX WORKS installation on an engineering PC.

Click on the  button in the header to start ctrlX PLC Engineering.

Upon the initial ctrlX PLC Engineering start, a project template is automatically created in the device tree that is compatible with the ctrlX hardware used. The project template already has an object structure in which an application is preconfigured. The application is loaded onto the ctrlX device when going online and is displayed after the project download in the “Applications” tab, see ➔ [Tab – “Applications”](#).

Additional applications can be added in the device tree of ctrlX PLC Engineering and loaded onto the ctrlX device.

Restrictions

The “child application” application variant is not supported.

13.3 Header

13.3.1 Header – “PLC”

The header in the “PLC” window is used to centrally control the PLC applications on the ctrlX device.

Functions

- Controlling the operating status of the applications
- Reset the applications
- Configuration of the retain data memory size
- Calling ctrlX PLC Engineering to create PLC projects

Call

In the ctrlX OS web interface:

“side navigation → PLC”

Description

Interface element	Description
	Start applications
	Stop applications
	Reset all applications with the following reset options: • Reset Warm • Reset Cold • Reset Origin
	Calling ctrlX PLC Engineering to create the PLC project
	Opens the “Allocate PLC retain memory (Bytes)” dialog to change the memory size, see ↗ Change the memory size for remanent data

Additional information

[↗ Window – “PLC”](#)

13.4 Tabs

13.4.1 Tab – “Applications”

The tab provides a tabular list of all PLC applications (application) that are configured or loaded on the ctrlX device. The current operating status and any pending diagnostic messages are displayed for each listed application. Buttons can be used to switch the operating status of individual applications or perform a reset, see options below.

Call

In the ctrlX OS web interface:

“side navigation → PLC → Tab “Applications””

Description

Table element	Description
“Name”	PLC application name

Table element	Description
"State"	Operating status of the PLC application: • "RUN": PLC application has been launched • "STOP": PLC application is stopped
"Diagnostics"	Display of pending diagnostic messages: • "Exception" = The application is in error state. Measure: For further details please open ctrlX PLC Engineering and check the "PLC Log and the "Call tree". • "ForceVariablesActive" = The "Force Values" function is active in the application. Measure: To end forcing, execute in ctrlX PLC Engineering the command "Unforce values". • "FlowControlActive" = The "Flow control" mode is active in the application. Measure: To disable the Flow control, execute in ctrlX PLC Engineering the "Flow control" command. • "RetainMismatch" = Application loaded with "retain mismatch" loaded. Measure: To run the application, a "Reset Cold" is required, see "Actions" below.
"Actions"	Buttons or operations for controlling the operating status: • 1. ▷ = Start application • 2. □ = Stop application • 3. ⌂ = Reset application with the following options: - 3.1 Reset warm - 3.2 Reset cold - 3.3 Reset origin Important information: From version PLC-V-0304, operations 2 and 3 are asynchronous services with a standard timeout of 30000 ms. The timeout is used to cancel the service if it cannot complete the task in time. Via the [plc/admin/runtime-settings/async-timeout-settings] Data Layer node, the value for the timeout can be adjusted if necessary, for example in case of high cycle times in the PLC program.

Related topics

➔ [Window – “PLC”](#)

13.4.2 Tab – “Boot applications”

The tab provides a tabular list of all boot applications that are configured on the ctrlX device and their time of creation.

Call

In the ctrlX OS web interface:

“side navigation → PLC → Tab “Boot applications””

Description

Table element	Description
"Name"	Boot application name
"Created"	Time of creation of the boot application (date/time)

Related topics

➔ [Window – “PLC”](#)

13.4.3 Tab – “Project information”

The tab provides general information about the PLC project in a tabular comparison display between the currently loaded project and the last loaded source project.

- Table column “Last Loaded”
Provides information from the last application loaded on the control
- Table column “Synchronization”
Provides information from the project sources last loaded onto the control

The comparison view contains the following information:

- Project name
- Last change date
- Details of the engineering PC / user name

Call

In the ctrlX OS web interface:

“side navigation → PLC → Tab “Project information””

Description

Table element	Table column “Last Loaded”	Table column “Synchronization”
“Name”	Name of the project on the ctrlX device	Name of the synchronized project
“Last change”	Date and time of the last changes to the applications of the target project.	Date and time of the last changes to the applications of the source project.
“Machine name”	Name of the engineering PC from which an application was downloaded.	
“User name”	User name of the engineering PC that was used to load the application.	

Related topics

➔ [Window – “PLC”](#)

14 Related documentation

14.1 Overview

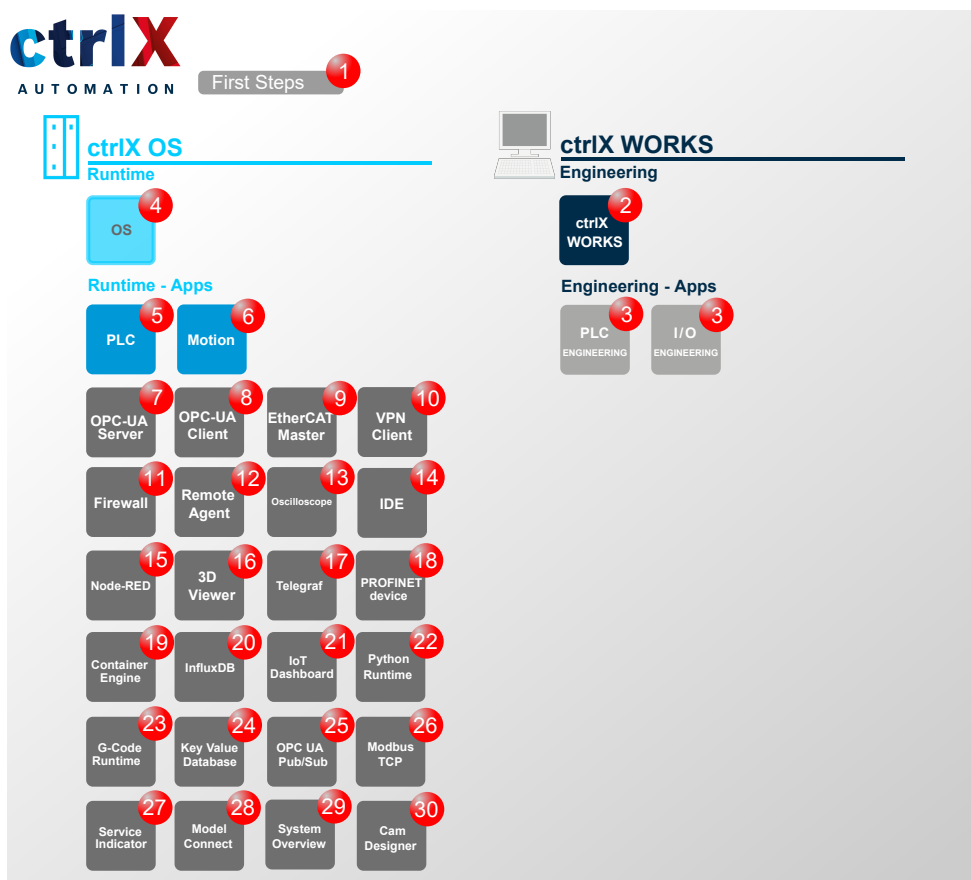


Fig. 15: Overview on further documentations

14.2 ctrlX AUTOMATION

No.	Documentation
1	ctrlX WORKS First Steps Quick Start Guide → Web documentation link Ordering information: • DOK-XWORKS-F*STEP*****-QU01-EN-P • R911403760

14.3 ctrlX WORKS

No.	Documentation
2	ctrlX WORKS Basic System 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XWORKS-WRK***V02**-APRS-EN-P • R911421576
3	ctrlX PLC Engineering - PLC Programming System 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XPLC**-ENG*****V02-APRS-EN-P • R911421578
3	ctrlX PLC Engineering - PLC Libraries 02VRS Reference Book ↗ Web documentation link Ordering information: • DOK-XPLC**-LIB***V02**-RERS-EN-P • R911421580

14.4 ctrlX OS

No.	Documentation
4	ctrlX OS - Operating System for ctrlX CORE Control Devices 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-XCR***V02**-APRS-EN-P • R911421590
	ctrlX OS - Data Layer nodes 02VRS Reference Book ↗ Web documentation link Ordering information: • DOK-XCORE*-DL****V02**-RERS-EN-P • R911421592
	ctrlX OS - Diagnostics 02VRS Reference Book ↗ Web documentation link Ordering information: • DOK-XCORE*-DIAG**V02**-RERS-EN-P • R911421594

14.5 ctrlX OS apps

No.	Documentation
5	PLC App - PLC Runtime Environment for ctrlX OS 02VRS Application Manual ↪ Web documentation link Ordering information: • DOK-XCORE*-PLC***V02**-APRS-EN-P • R911421584
6	Motion App - Motion Runtime Environment for ctrlX CORE 02VRS Application Manual ↪ Web documentation link Ordering information: • DOK-XCORE*-MOT***V02**-APRS-EN-P • R911421610
7	OPC UA Server App - OPC UA Server for ctrlX OS 02VRS Application Manual ↪ Web documentation link Ordering information: • DOK-XCORE*-UAS***V02**-APRS-EN-P • R911421598
8	OPC UA Client App - OPC UA Client for ctrlX OS 02VRS Application Manual ↪ Web documentation link Ordering information: • DOK-XCORE*-UAC***V02**-APRS-EN-P • R911421600
9	EtherCAT Master App - EtherCAT master for ctrlX OS 02VRS Application Manual ↪ Web documentation link Ordering information: • DOK-XCORE*-ECM***V02**-APRS-EN-P • R911421604
10	VPN Client App - Remote Support Software for ctrlX OS 02VRS Application Manual ↪ Web documentation link Ordering information: • DOK-XCORE*-VPN***V02**-APRS-EN-P • R911421596
11	Firewall App - Security Functions for ctrlX OS 02VRS Application Manual ↪ Web documentation link Ordering information: • DOK-XCORE*-FRW***V02**-APRS-EN-P • R911421606

No.	Documentation
12	Remote Agent App - ctrlX Device Portal Connection for ctrlX OS Devices 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-RMA***V02**-APRS-EN-P • R911421608
13	Oscilloscope App - Oscilloscope Function for ctrlX OS Devices 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-OSC***V02**-APRS-EN-P • R911421587
14	IDE App - Integrated Development Environment 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-IEN***V02**-APRS-EN-P • R911421612
15	Node RED App - Graphic Programming for ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-NOENRED*V02-APRS-EN-P • R911421582
16	3D Viewer App - Browser-based 3D Kinematic Simulation for ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-3DV***V02**-APRS-EN-P • R911421615
17	Telegraf App - Server Agent for Collecting Data in the Data Layer 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-TSA***V02**-APRS-EN-P • R911421623
18	PROFINET Device App - PROFINET Device for ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-PROFINETV02-APRS-EN-P • R911421617

No.	Documentation
19	Container Engine App - Using Docker® Images on the ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-DOE***V02**-APRS-EN-P • R911421619
20	InfluxDB App - Influx Database Connection for ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-IDB***V02**-APRS-EN-P • R911421625
21	IoT Dashboard App - Data Visualization in Dynamic, Interactive Dashboards 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-GDB***V02**-APRS-EN-P • R911421633
22	Python Runtime App - Python Runtime App Environment for ctrlX CORE 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-PYR***V02**-APRS-EN-P • R911421629
23	G-Code Runtime App - G-Code Interpreter for ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-GCO***V02**-APRS-EN-P • R911421631
24	Key Value Database App - Data Layer Management 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-KVD*****V02-APRS-EN-P • R911418735
25	OPC UA Pub/Sub App - OPC UA Pub/Sub for ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-UAP***V02**-APRS-EN-P • R911421602

No.	Documentation
26	Modbus TCP App - Modbus TCP Communication over ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-MOD*TCP*V02-APRS-EN-P • R911421621
27	Service Indicator App Service Indicator for ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-SIN*****V02-APRS-EN-P • R911421627
28	Model Connect App Target for Model-based Development and Simulation for ctrlX OS 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-MOC***V02**-APRS-EN-P • R911421631
29	System Overview App - System Topology and System Information 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XCORE*-SOV***V02**-APRS-EN-P • R911424409
30	Cam Designer - Configuring ctrlX Motion Cams 02VRS Application Manual ↗ Web documentation link Ordering information: • DOK-XWORKS-CAM***V02**-APRS-EN-P • R911424390

15 Service and support

Our worldwide service network provides an optimized and efficient support. Our experts provide you with advice and assistance. You can contact us **24/7**.

Service Germany

Our technology-oriented Competence Center in Lohr, Germany, is responsible for all your service-related queries for electric drive and controls.

Contact the **Service Hotline** and **Service Helpdesk** under:

Phone: **+49 9352 40 5060**

Fax: **+49 9352 18 4941**

Email: ➔ service.svc@boschrexroth.de

Internet: ➔ <http://www.boschrexroth.com>

Additional information on service, repair (e.g. delivery addresses) and training can be found on our internet sites.

Service worldwide

Outside Germany, please contact your local service office first. For hotline numbers, refer to the sales office addresses on the internet.

Preparing information

To be able to help you more quickly and efficiently, please have the following information ready:

- Detailed description of malfunction and circumstances
- Type plate specifications of the affected products, in particular type codes and serial numbers
- Your contact data (phone and fax number as well as your e-mail address)

16 Index

C

ctrlX AUTOMATION

Related documentation 48

H

Header

PLC 45

Helpdesk 54

Hotline 54

I

Intended use

Areas of application 6

Areas of use 6

Introduction 5

L

Licensing the multicore functionality . . . 15, 16

M

Multicore introduction 14

P

PLC App

Basics 7, 10

Data Layer connection 13

Flexible configuration of the memory for
remanent PLC data 22

Side navigation 42

Tab "Applications" 45

Tab "Boot applications" 46

Tab "Project information" 47

Target systems 12

PLC runtime system

Connection to the ctrlX OS system behavior
. 23

PLC task

External event-controlled PLC tasks 40

Freely configurable, external event sources
. 41

S

Safety instructions 7

Service hotline 54

Support 54

U

Unintended use 6

Consequences, disclaimer 6

W

Window

PLC 44

Bosch Rexroth AG
Bgm.-Dr.-Nebel-Str. 2
97816 Lohr a.Main
Germany
Tel. +49 9352 18 0
Fax +49 9352 18 8400
www.boschrexroth.com/electrics



R911423401 03